

IX. Les sélecteurs CSS avancés — Cibler précisément

LES SÉLECTEURS EN UNE PHRASE

Un sélecteur désigne **QUELS ÉLÉMENTS HTML** vont recevoir un style — et plus on est précis, plus on a de contrôle.

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Sélecteurs simples (rappel)**
- **B. Sélecteurs combinés**
 - 1. Descendant (, espace)
 - 2. Enfant direct
 - 3. Frère adjacent (+)
 - 4. Frère général (~)
 - 5. Récapitulatif visuel
- **C. Sélecteurs d'attribut**
 - Tableau des opérateurs
- **D. Pseudo-classes**
 - 1. États d'interaction
 - 2. Position dans les éléments frères
 - 3. La pseudo-classe :not()
 - 4. Validation de formulaire
- **E. Pseudo-éléments**
 - 1. ::before et ::after
 - 2. Autres pseudo-éléments utiles
- **F. Grouper les sélecteurs**
 - Anti-pattern : sur-grouper
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Distinguer** les différents types de sélecteurs (type, classe, id, attribut, pseudo-classe, pseudo-élément).
- **Combiner** plusieurs sélecteurs (descendant, enfant direct, frère adjacent, frère général).
- **Utiliser** les pseudo-classes courantes (`:hover` , `:focus` , `:nth-child` , `:not`) pour enrichir l'interaction.
- **Appliquer** les pseudo-éléments `::before` et `::after` pour ajouter du contenu sans toucher au HTML.
- **Évaluer** la complexité d'un sélecteur et en choisir un plus simple quand possible.

2. En pratique — situation professionnelle

Le designer vous demande : *le premier item de la liste doit être rouge, les items impairs en gras, et seuls les liens externes doivent avoir une icône devant*. Sans maîtrise des sélecteurs avancés (pseudo-classes, sélecteurs d'attribut, pseudo-éléments), vous modifiez le HTML en ajoutant des classes partout. Cette leçon vous montre comment le faire **sans toucher au HTML**.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Styliser sans modifier le HTML (<code>:hover</code> , <code>[type="email"]</code>)	Sélecteurs > 3 niveaux : ajouter une classe à la place
<code>:nth-child()</code> pour alterner les couleurs de lignes	Pseudo-élément <code>::before</code> sans <code>content</code> (ignoré)
<code>::before</code> / <code>::after</code> pour décorations (icônes, citations)	Reproduire du contenu sémantique en <code>::before</code> (<code>ally</code>)

A. Sélecteurs simples (rappel)

Vus au chapitre II, mais résumés ici pour référence.

Syntaxe	Sélectionne	Exemple
<code>*</code>	Tous les éléments (sélecteur universel)	<code>* { margin: 0; }</code>
<code>element</code>	Tous les éléments de ce type	<code>p { color: blue; }</code>
<code>.classe</code>	Tous les éléments avec la classe	<code>.btn { ... }</code>
<code>#id</code>	L'élément avec cet id unique	<code>#header { ... }</code>

B. Sélecteurs combinés

Les **combinateurs** relient plusieurs sélecteurs pour cibler des éléments en fonction de leur **position** dans l'arborescence HTML.

1. Descendant (, espace)

Cible **B** à **n'importe quelle profondeur** à l'intérieur de **A**.

```
article p {  
  font-style: italic;  
}
```

→ Tous les **<p>** **imbriqués** quelque part dans un **<article>**, même à plusieurs niveaux de profondeur.

2. Enfant direct

Cible **B** **seulement si enfant immédiat** de **A**.

```
nav > ul {  
  list-style: none;  
}
```

→ Le **** doit être **directement** dans **<nav>** (pas enveloppé dans un autre élément).

3. Frère adjacent (+)

Cible **B** **seulement si placé juste après** **A** (même parent).

```
h1 + p {  
  font-weight: bold;  
}
```

→ Le **premier** **<p>** qui suit **immédiatement** un **<h1>** dans le même parent.

4. Frère général (~)

Cible **tous les** **B** qui suivent **A** dans le même parent.

```
h1 ~ p {  
  margin-top: 1rem;  
}
```

→ **Tous** les `<p>` qui viennent après un `<h1>` (même s'ils ne sont pas adjacents).

5. Récapitulatif visuel

```
<article>  
  <h1>Titre</h1>  
  <p>Para A</p>    <!-- ← ciblé par h1 + p ET h1 ~ p -->  
  <p>Para B</p>    <!-- ← ciblé par h1 ~ p seulement -->  
  <section>  
    <p>Para C</p> <!-- ← ciblé par article p seulement -->  
  </section>  
</article>
```

♦ Quelle combinaison choisir ?

- **Descendant** (`>`) est le plus simple — utilisez-le sauf raison de faire autrement.
- **Enfant direct** (`>`) quand vous voulez limiter à un niveau précis (ex: menu de navigation où `nav > ul > li` cible les items principaux, pas les sous-menus).
- `+ et ~` sont plus rares mais très puissants pour des mises en forme basées sur l'ordre.

C. Sélecteurs d'attribut

Cibler un élément selon la **présence ou la valeur** d'un attribut HTML.

```
/* Tous les inputs de type email */
input[type="email"] {
  border-color: #0066cc;
}

/* Tous les liens externes (avec https) */
a[href^="https://"] {
  color: darkblue;
}

/* Tous les liens qui terminent par .pdf */
a[href$=".pdf"] {
  font-style: italic;
}

/* Tous les éléments contenant "bouton" dans leur attribut title */
[title*="bouton"] {
  cursor: pointer;
}
```

Tableau des opérateurs

Syntaxe	Sens
[attr]	A l'attribut (quelle que soit sa valeur)
[attr="valeur"]	Égal à valeur
[attr^="val"]	Commence par val
[attr\$="val"]	Se termine par val
[attr*="val"]	Contient val
[attr~="val"]	Contient val comme mot entier (liste séparée par espaces)

❗ Sélecteurs d'attribut = alternative aux classes

Ils permettent de styliser des éléments **sans ajouter de classe** au HTML. Très utile pour styliser selon le type d'input ou la destination d'un lien.

D. Pseudo-classes

Les **pseudo-classes** ciblent un élément selon son **état** ou sa **position**. Elles commencent par un seul `:`.

1. États d'interaction

```
a:hover {  
  color: red;  
}  
  
input:focus {  
  outline: 2px solid blue;  
}  
  
button:disabled {  
  opacity: 0.5;  
  cursor: not-allowed;  
}
```

- `:hover` — l'utilisateur survole avec la souris.
- `:focus` — l'élément a le focus clavier.
- `:active` — pendant le clic.
- `:disabled` / `:enabled` — état du champ de formulaire.
- `:checked` — case/radio cochée.
- `:visited` — lien déjà visité par l'utilisateur.

2. Position dans les éléments frères

```
li:first-child { font-weight: bold; }  
li:last-child { border-bottom: 0; }  
li:nth-child(odd) { background: #f4f4f4; } /* impairs */  
li:nth-child(even) { background: white; } /* pairs */  
li:nth-child(3) { color: red; } /* le 3e */  
li:nth-child(3n) { color: green; } /* 3, 6, 9, ... */
```

3. La pseudo-classe `:not()`

Exclure certains éléments :


```
button:not(.btn-disabled) {  
  cursor: pointer;  
}  
  
input:not([type="hidden"]) {  
  display: block;  
}
```

4. Validation de formulaire

```
input:required { border-left: 3px solid orange; }  
input:valid    { border-color: green; }  
input:invalid  { border-color: red; }
```

→ S'appuie sur les attributs HTML5 (`required` , `pattern` , `type="email"` , ...) sans JavaScript.

E. Pseudo-éléments

Les **pseudo-éléments** permettent de styliser **une partie d'un élément** ou d'**ajouter du contenu virtuel**. Ils commencent par **deux** `:` (`::`).

1. `::before` et `::after`

Ajoutent un pseudo-contenu avant ou après l'élément. **Requièrent toujours** `content:`.

```
/* Ajouter une icône devant les liens externes */
a[target="_blank"]::after {
  content: " ↗";
  color: gray;
}

/* Citation automatique */
blockquote::before {
  content: "« ";
  font-size: 2em;
  color: #ccc;
}
blockquote::after {
  content: " »";
  font-size: 2em;
  color: #ccc;
}
```

2. Autres pseudo-éléments utiles

```
p::first-line {  
  font-weight: bold;  
}  
  
p::first-letter {  
  font-size: 2em;  
  float: left;  
  margin-right: 0.5rem;  
}  
  
::selection {  
  background: yellow; /* couleur du texte sélectionné par l'utilisateur */  
}  
  
::placeholder {  
  color: #999;  
  font-style: italic;  
}
```

⚠️ `:` vs `::`

- **1 seul `:`** = pseudo-**classe** (état : `:hover` , `:focus`).
- **2 `::`** = pseudo-**élément** (partie virtuelle : `::before` , `::after`).

Historiquement, `:before` et `:after` sans deux-points fonctionnent encore pour compatibilité. En 2026, **toujours écrire** `::` pour être explicite.

F. Grouper les sélecteurs

Plusieurs sélecteurs séparés par une **virgule** partagent les mêmes déclarations.

```
h1, h2, h3 {  
  font-family: "Georgia", serif;  
  color: #222;  
}  
  
input[type="text"],  
input[type="email"],  
input[type="password"],  
textarea {  
  padding: 8px;  
  border: 1px solid #ccc;  
}
```

Bénéfice : éviter la répétition. Un seul bloc pour trois propriétés.

Anti-pattern : sur-grouper

```
/* MAL : trop de sélecteurs, illisible */  
.header .nav ul li a,  
.footer .nav ul li a,  
.sidebar ul li a,  
aside nav a {  
  color: #333;  
}
```

◆ Règle du pouce

Si un sélecteur fait plus de **3 niveaux** de profondeur (**a b c d**), c'est un signe qu'il faut **simplifier** — soit en ajoutant une classe au bon endroit, soit en revoyant la structure HTML.

Conclusion

1. À retenir

- **Combinateurs** : `A B` (descendant), `A > B` (enfant direct), `A + B` (frère adjacent), `A ~ B` (tous frères suivants).
- **Sélecteurs d'attribut** : `[type="email"]`, `[href^="https://"]`, `[href$=".pdf"]`.
- **Pseudo-classes** (`:hover`, `:nth-child(n)`, `:not(.x)`) ciblent des **états** ou **positions**.
- **Pseudo-éléments** (`::before`, `::after`) ajoutent du **contenu virtuel** — requièrent `content:`.
- **Virgule** pour grouper plusieurs sélecteurs partageant les mêmes styles.
- Éviter les sélecteurs à plus de 3 niveaux de profondeur : ajoutez une classe au bon endroit.

2. Pour aller plus loin

- **MDN — Sélecteurs CSS (guide complet)**
- **MDN — Pseudo-classes**
- **MDN — Pseudo-éléments**
- **CSS Selectors Cheatsheet**
- **CSS Diner** — jeu gamifié pour s'entraîner aux sélecteurs (32 niveaux progressifs).